Bibliotech — Class Diagram and Component Model

Project: Bibliotech
Document: Domain, application, infrastructure, adapter and frontend class model
Generated: 2026-10-27
Sources of truth: `bibliotech/backend/domain/`, `bibliotech/backend/application/`, `bibliotech/backend/infrastructure/`, `bibliotech/backend/controllers/`, `bibliotech/backend/routes/`, `bibliotech/backend/middleware/`, `bibliotech/backend/validator/`, `bibliotech/frontend/js/`

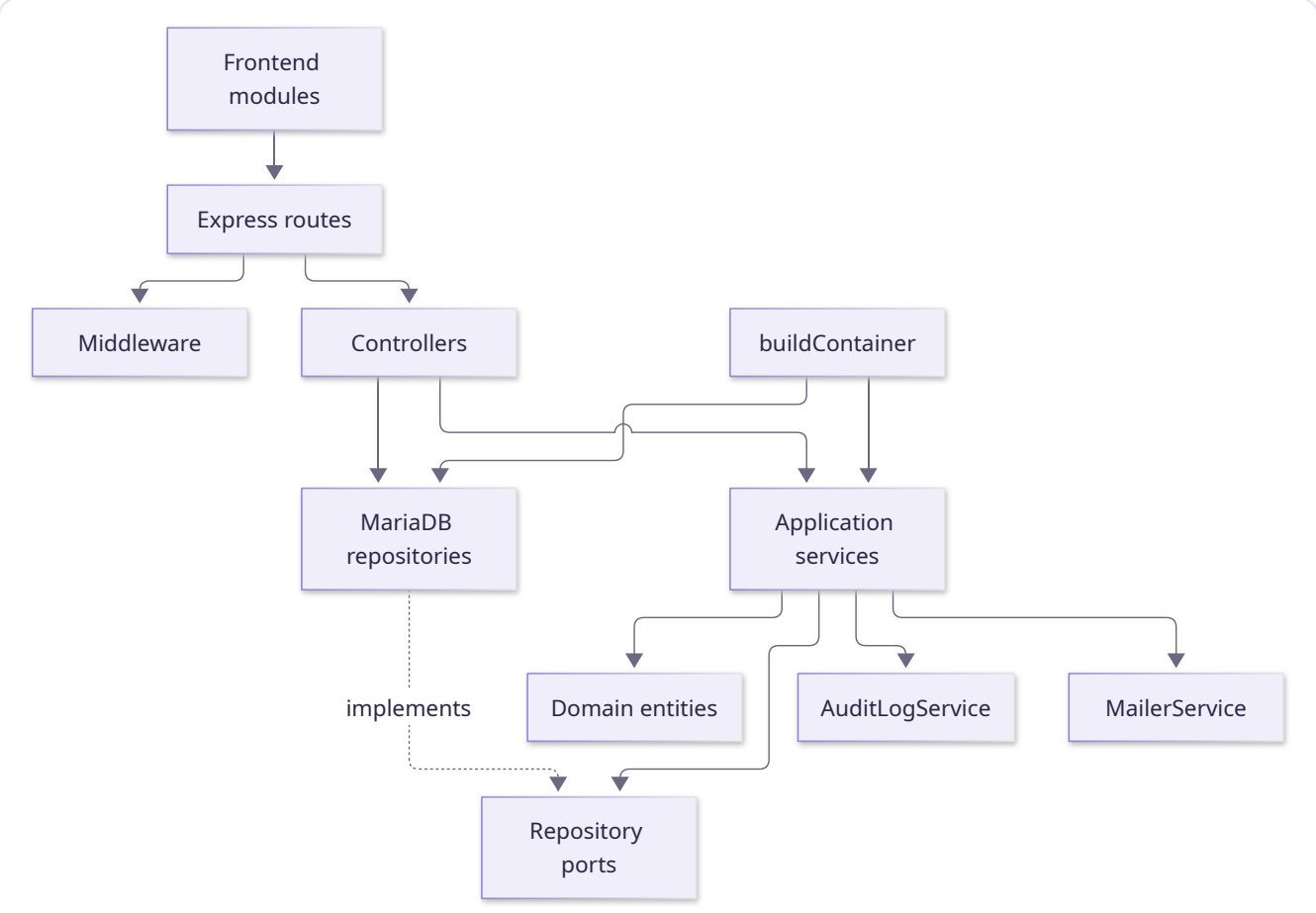
The backend is implemented with CommonJS modules. Classes are used for domain entities, ports, services, repositories, validators, middleware helpers and frontend components. Express controllers and routes are factory functions/modules; they are shown with the `<<factory>>` or `<<router>>` stereotype. The diagrams are split by layer to keep each view readable.

1. Architectural Overview

Bibliotech follows a **hexagonal / ports-and-adapters** structure:

Layer	Responsibility	Main files
Domain	Business entities, observer pattern, repository contracts	`domain/entities/`, `domain/observers/`, `domain/ports/`
Application	Use cases, authorization, moderation, audit and notification orchestration	`application/services/`
Infrastructure	MariaDB repositories, email transport, file helpers, dependency container	`infrastructure/`, `utils/`
Input adapters	Express routes, controllers, middleware, validators, view routes	`routes/`, `controllers/`, `middleware/`, `validator/`
Frontend	Page controllers, API client, Web Components, i18n, validation	`frontend/js/`

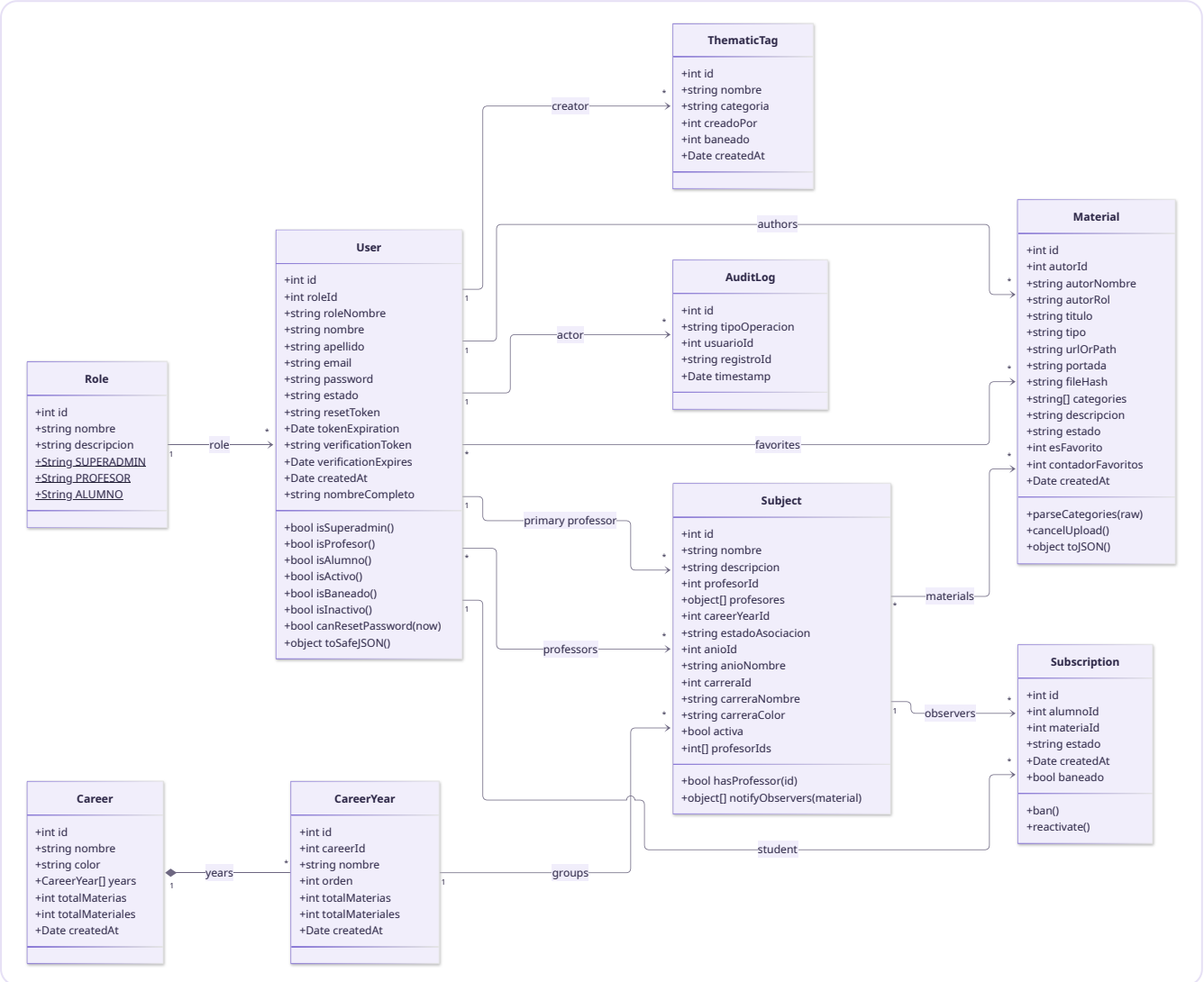
Dependency direction



Key rule: application services depend on domain ports, never on concrete MariaDB classes. Concrete repositories are injected by `infrastructure/container.js`. A few read-only controllers also use repositories directly for list operations, which is a pragmatic shortcut rather than a violation of the domain flow.

2. Domain Layer

2.1 Domain entities

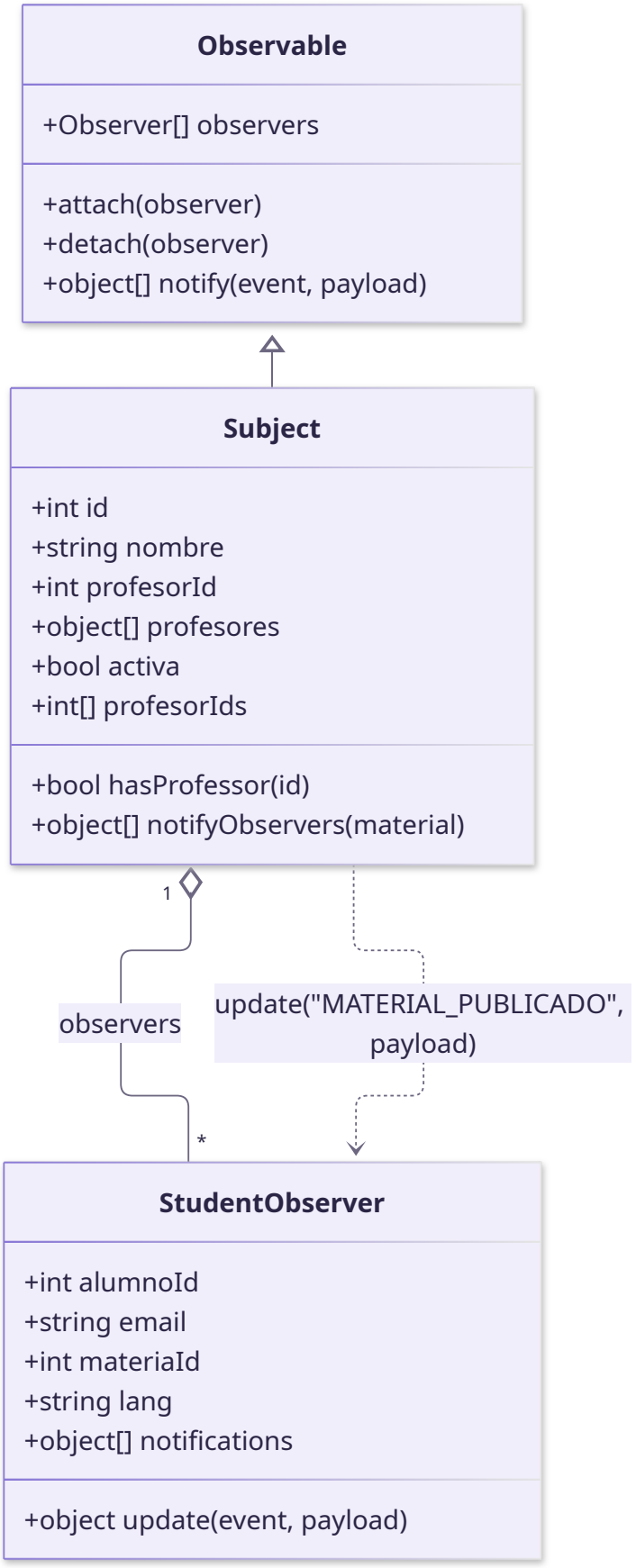


Notes

- `Subject` also inherits from `Observable`; the inheritance is shown in the observer diagram.
- `Material.TIPOS` is a static list: `PDF`, `EPUB`, `Word`, `Presentacion`, `Link_Drive`, `Youtube`, `Repositorio`.
- `Material.cancelUpload()` only enforces the “pending only” rule and changes the in-memory state; persistence is completed by `MaterialService` and `MaterialRepository`.
- `Subscription.ban()` and `Subscription.reactivate()` change the in-memory state; persistence is completed by the repository/service.
- `User.toSafeJSON()` removes `password`, reset token data and verification token data before API serialization.

- There is no domain entity class for notifications. The notification service and repository use plain row objects.

2.2 Observer pattern

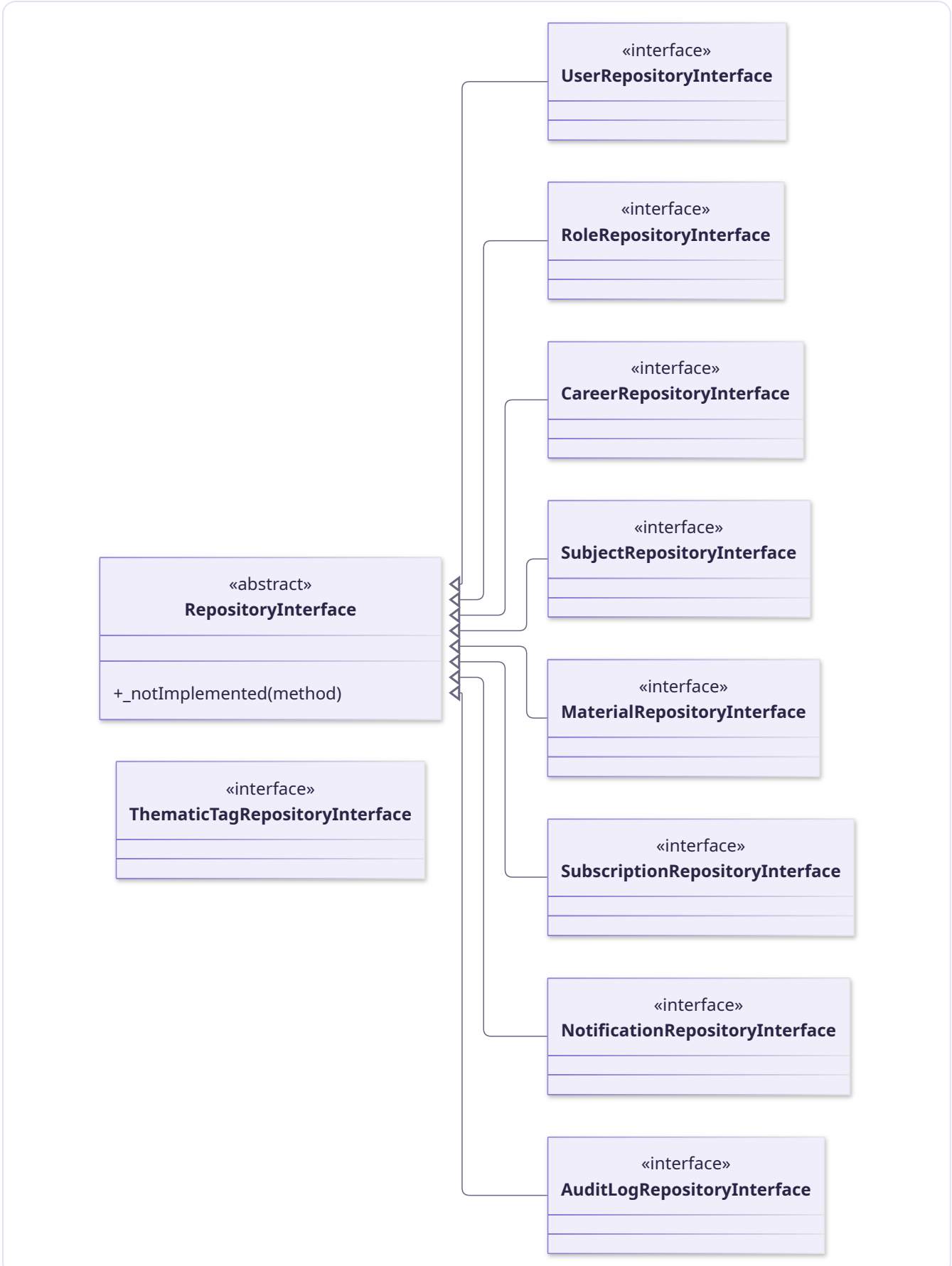


Runtime behavior

1. `MaterialService.notifyObservers(subject, material)` loads all `Suscrito` subscribers from `subscriptionRepository.getObservers(subject.id)`.
2. It creates a transient `Subject` instance and attaches one `StudentObserver` per subscriber row.
3. `Subject.notifyObservers(material)` calls `notify('MATERIAL_PUBLICADO', ...)`.
4. Each `StudentObserver.update()` returns an in-memory notification object.
5. The service separately persists notifications through `NotificationRepository.createForSubscribers()`. Persistence failure is logged but does not abort the material upload/approval.

Design consequence: the Observer pattern is used as the domain notification mechanism, while the `notifications` table is the durable inbox. In-memory observer notifications are ephemeral and are not returned by the API.

3. Repository Ports

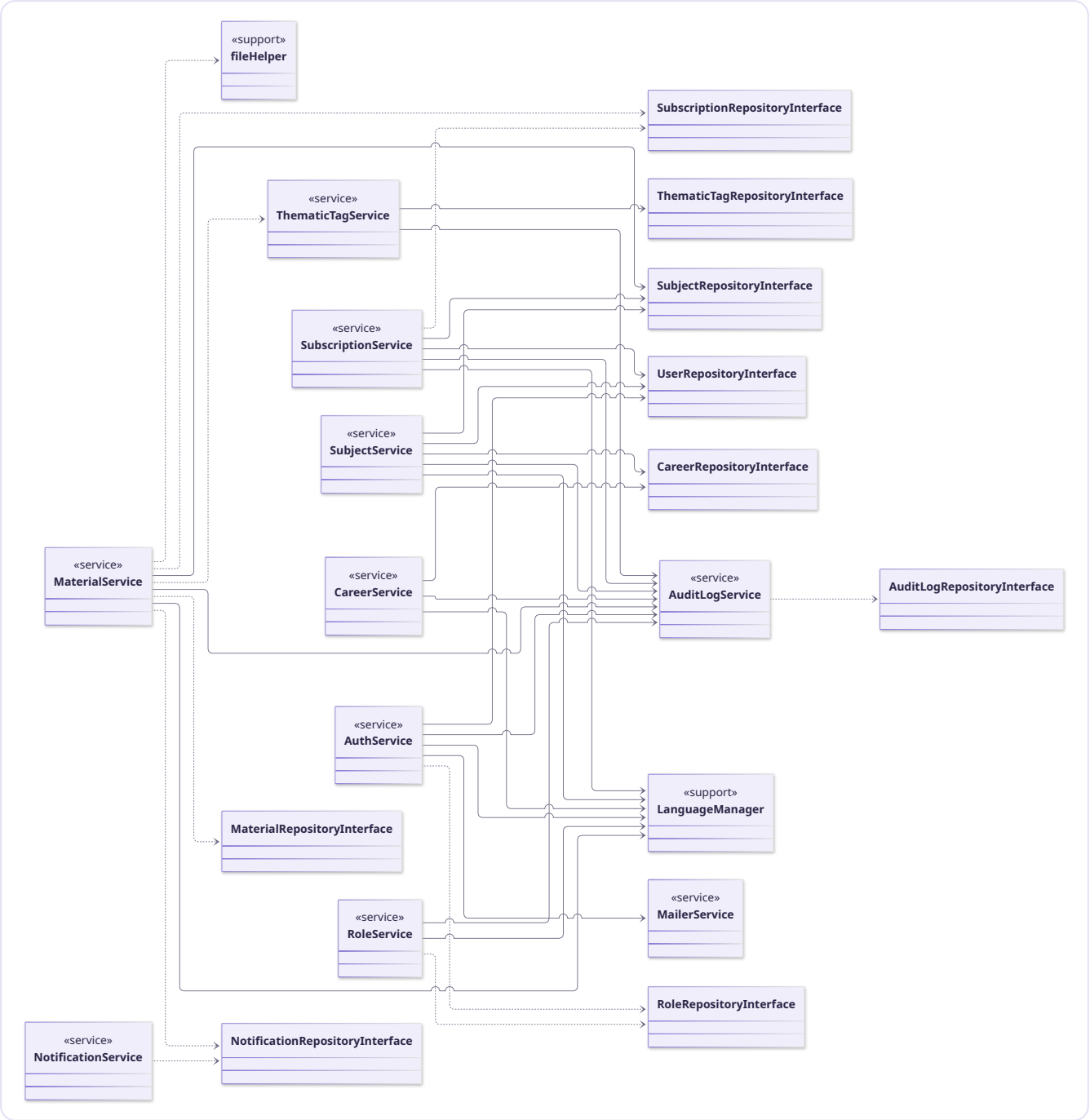


`ThematicTagRepositoryInterface` is the one exception: it does **not** extend `RepositoryInterface`; it throws its own “Not implemented” errors. This is a small inconsistency in the current codebase.

Port contracts

Port	Methods
`RepositoryInterface`	`_notImplemented(method)`
`UserRepositoryInterface`	`save`, `findById`, `findByEmail`, `updatePassword`, `setResetToken`, `findByResetToken`, `setVerificationToken`, `findByVerificationToken`, `verifyUser`, `updateEstado`, `softDelete`, `findAll`, `findByRole`
`RoleRepositoryInterface`	`save`, `findById`, `findByName`, `findAll`, `delete`
`CareerRepositoryInterface`	`save`, `update`, `findById`, `findAll`, `saveYear`, `updateYear`, `findYearById`, `findYearsByCareer`
`SubjectRepositoryInterface`	`save`, `findById`, `findAll`, `update`, `addProfessor`, `removeProfessor`, `replaceProfessors`, `requestAssociation`, `confirmAssociation`, `rejectAssociation`, `delete`
`MaterialRepositoryInterface`	`save`, `findById`, `findByHash`, `findAllGlobal`, `findBySubject`, `findByAuthor`, `findPendingByAuthor`, `findPendingForProfessor`, `countPendingByAuthor`, `associateToSubject`, `updateAssociationState`, `updateEstado`, `deleteAssociation`, `countAssociations`, `cancelPendingUpload`, `rejectPendingUpload`, `update`, `delete`, `toggleFavorite`, `findFavorites`, `findFavoriteIds`, `countFavorites`
`SubscriptionRepositoryInterface`	`save`, `findById`, `findByAlumnoAndMateria`, `subscribe`, `banStudent`, `getObservers`, `findByAlumno`, `delete`
`NotificationRepositoryInterface`	`createForSubscribers`, `findByUser`, `countUnread`, `markRead`, `markAllRead`
`AuditLogRepositoryInterface`	`save`, `findAll`, `findByUser`
`ThematicTagRepositoryInterface`	`findAll`, `findById`, `findByNameCategory`, `save`, `ban`, `delete`

4. Application Services



Service catalog

Service	Responsibility	Main methods	Audit / side effects
`AuthService`	Registration, professor creation, login, verification, password reset, ban/unban/soft-delete	`register`, `createProfessor`, `login`, `requestPasswordReset`, `activateAccount`, `verifyAccount`, `resendVerification`, `validateResetToken`, `resetPassword`, `banUser`, `unbanUser`, `softDeleteUser`, `getUserById`	Audits user lifecycle; sends verification/recovery email; hashes passwords and tokens with bcrypt.
`RoleService`	Role lookup and creation	`createRole`, `getRoleById`, `getAllRoles`	Audits role creation as `Alta Rol`; validates uniqueness.
`CareerService`	Careers and year levels	`createCareer`, `updateCareer`, `createYear`, `updateYear`, `getAllCareers`, `getCareerById`	Audits `Alta Carrera`, `Modificacion Carrera`, `Alta Anio`, `Modificacion Anio`; validates color and update order (create year coerces order without a minimum check).
`SubjectService`	Subject CRUD, professor assignments, association requests	`createSubject`, `requestAssociation`, `confirmAssociation`, `rejectAssociation`, `getAllSubjects`, `getSubjectById`, `updateSubject`	Audits `Alta Materia`, `Modificacion Materia`, `Asociacion Materia`, `Solicitud Materia`; verifies professor roles.
`MaterialService`	Upload/suggestion, duplicate handling, moderation, editing, deletion, batches, favorites, catalog queries	`uploadOrSuggestMaterial`, `associateMaterialToSubjects`, `approveMaterial`, `getPendingForProfessor`, `rejectMaterial`, `getPendingMaterials`, `cancelPendingUpload`, `update`, `delete`, `uploadBatch`, `getMyUploads`, `getByAuthor`, `getGlobalContent`, `getBySubject`, `toggleFavorite`, `getFavorites`	Audits upload/approval/cancel/delete/association; stores files; computes SHA-256; notifies subscribers; enforces ownership and pending limits.
`SubscriptionService`	Student subscriptions and observer bans	`subscribe`, `unsubscribe`, `banStudent`, `getMySubscriptions`, `getObservers`	Audits `Suscripcion Materia` and `Baneo Alumno`; checks subject ownership for bans.
`NotificationService`	Inbox read model	`list`, `markRead`, `markAllRead`	No audit.

Service	Responsibility	Main methods	Audit / side effects
`AuditLogService`	Append-only audit trail	`record`, `getAllLogs`, `getByUser`	No audit of itself.
`ThematicTagService`	Tag catalog and moderation	`list`, `create`, `ban`, `remove`	Rejects banned tags on creation and returns an existing non-banned tag; audit service is injected but not currently used.
`MailerService`	Gmail account rotation and email delivery	`send`	Uses `AccountManager` and `EmailDispatcher`; tries configured accounts; no audit.

Important service rules

`AuthService`

- Registration always creates an `Alumno` account in `PendienteVerificacion`.
- Professor creation is an admin action and creates an `Activo` professor.
- Login rejects `PendienteVerificacion`, `Inactivo` and `Baneado`.
- Password-reset requests are deliberately uniform to avoid account enumeration.
- Reset tokens are hashed in the database and expire.
- `toSafeJSON()` is used for API responses.

`SubjectService`

- `resolveProfessorIds()` accepts `profesorId` or `profesorIds`, verifies each user exists and has the `Profesor` role. This check is used by create/update; `confirmAssociation()` does not re-check the role when an admin supplies `profesorId`.
- `createSubject` can create a subject with no professor; it always starts with `estadoAsociacion = 'Activa'`. A later professor `requestAssociation` changes it to `'Pendiente'` until an admin confirms.
- `requestAssociation` adds the professor and marks the subject `Pendiente`.
- `confirmAssociation` adds the professor (if supplied) and marks the subject `Activa`.
- `rejectAssociation` removes all professors and clears the cached `profesor_id`; it sets `estadoAsociacion = 'Pendiente'` only when the subject was already pending, so an active subject can remain `Activa` while unassigned.

`MaterialService`

- Professor uploads are auto-approved and become visible immediately.
- Student uploads are pending and limited to three per student.
- Duplicate file hash:
  - Professor: associates the existing material with the selected subjects as `Aprobado`, without changing the material's global state.
  - Student: associates the existing material as pending.
- `assertCanManage()` allows:
  - the author at the service level, regardless of role (the `PUT /api/materials/:id` route itself allows only Professor/Superadmin);
  - a Superadmin, but only for a material whose author is an `Alumno` (because the non-student-author check runs first);

    - a professor, but only for a student suggestion in a subject the professor teaches.
- No caller can edit another professor's or administrator's material except its original author.
- `approveMaterial` and `rejectMaterial` require the caller to be a professor of the target subject (or admin).
- `delete(materialId, ctx, subjectId)` with a `subjectId` removes only that association; without one it deletes globally and is admin-only at the route level.
- Favorites are toggled with a read-then-write sequence (no explicit DB transaction) and return `{ favorited, contadorFavoritos }`; the returned count comes from a fresh `COUNT(*)`.

`SubscriptionService`

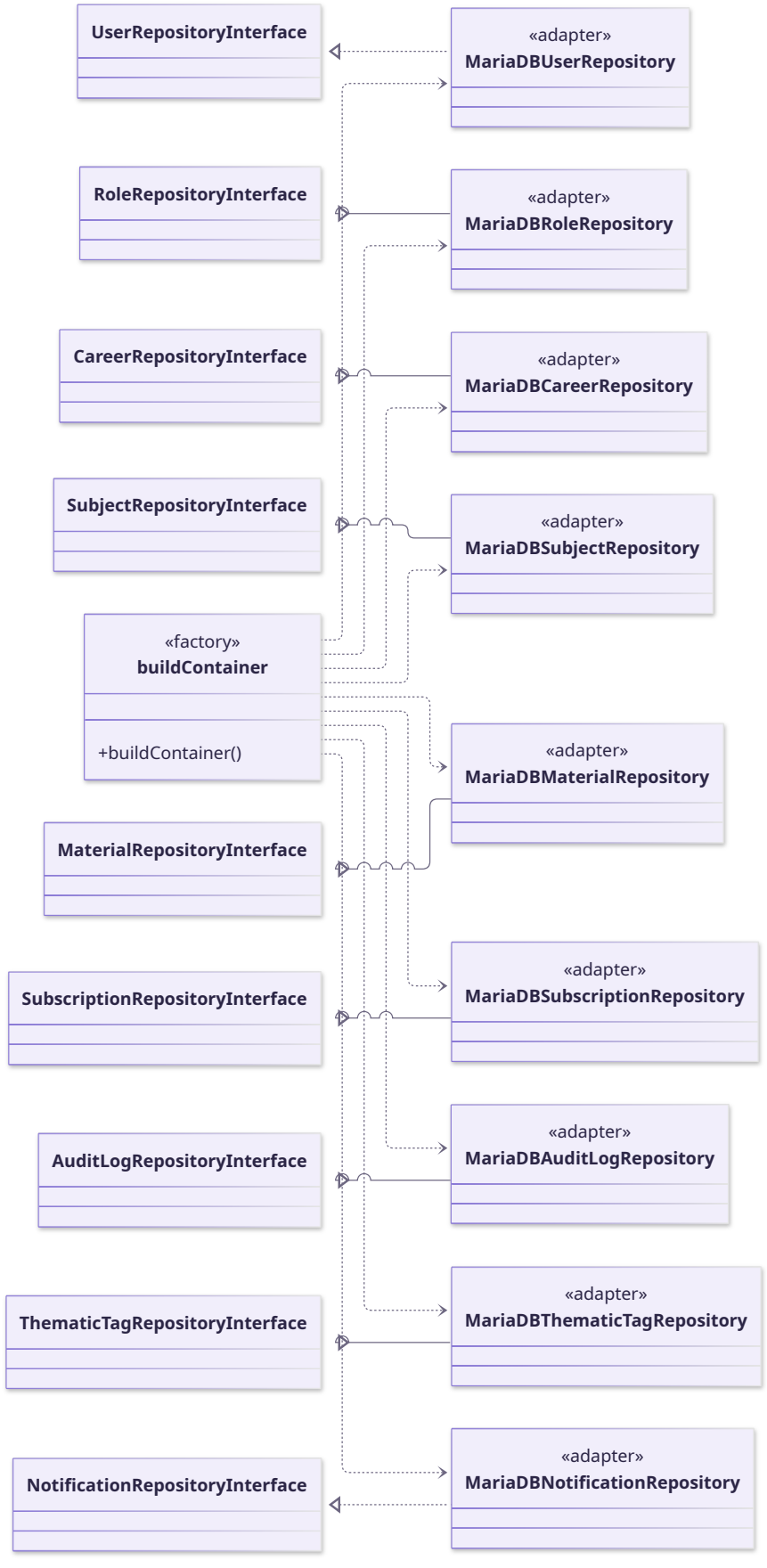
- Banning requires the caller to teach the subject (or be admin).
- Banned students cannot re-subscribe while the banned row exists.
- `getObservers` returns only `Suscrito` rows. The service does not check ownership for viewing the list; the route only requires Professor/Admin.

Role string constants

`Role.SUPERADMIN = 'Superadmin'`, `Role.PROFESOR = 'Profesor'`, `Role.ALUMNO = 'Alumno'`. These values match the database seed and the middleware authorization checks.

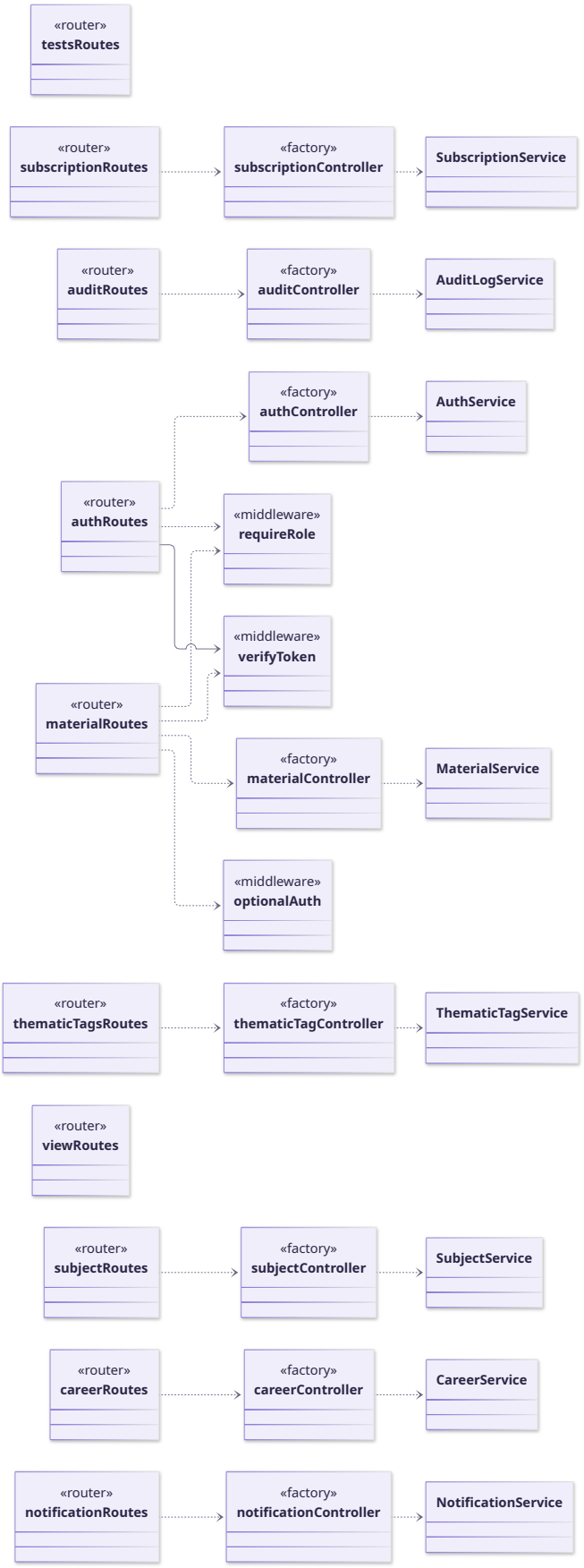
5. Infrastructure and Adapters

5.1 Dependency injection and repositories



Repository	Tables	Notable behavior
`MariaDBUserRepository`	`users`, `roles`	Joins role name; maps snake_case columns; handles verification/reset tokens; soft delete.
`MariaDBRoleRepository`	`roles`	Simple CRUD.
`MariaDBCareerRepository`	`careers`, `career_years`, `subjects`, `material_subjects`	Computes career/year totals; exposes years.
`MariaDBSubjectRepository`	`subjects`, `subject_professors`, `users`, `career_years`, `careers`, `material_subjects`	Attaches professor lists; synchronizes `subjects.profesor_id`; association request/confirm/reject.
`MariaDBMaterialRepository`	`materials`, `material_subjects`, `subjects`, `careers`, `career_years`, `users`, `roles`, `subject_professors`, `material_favorites`	Catalog filters, duplicate hash lookup, pending queues, favorite toggling, career metadata.
`MariaDBSubscriptionRepository`	`subscriptions`, `users`, `subjects`	Subscription upsert, observer list, ban, student list. `findById` / `findByAlumnoAndMateria` pass `SELECT *` rows into `Subscription`, so snake columns are not camelized; `findByAlumno` returns raw rows.
`MariaDBAuditLogRepository`	`audit_logs`, `users`, `subjects`, `materials`, `roles`, `careers`, `career_years`	Enriches audit rows with actor and target labels.
`MariaDBThematicTagRepository`	`thematic_tags`	Tag CRUD and ban. Raw snake_case rows are passed directly to `ThematicTag`, so `creado_por` / `created_at` are not mapped to `creadoPor` / `createdAt`.
`MariaDBNotificationRepository`	`notifications`, `subjects`, `materials`, `users`	Upserts per subscriber, unread counts, mark read; joins the subject professor for the inbox label.

5.2 Controllers and routes



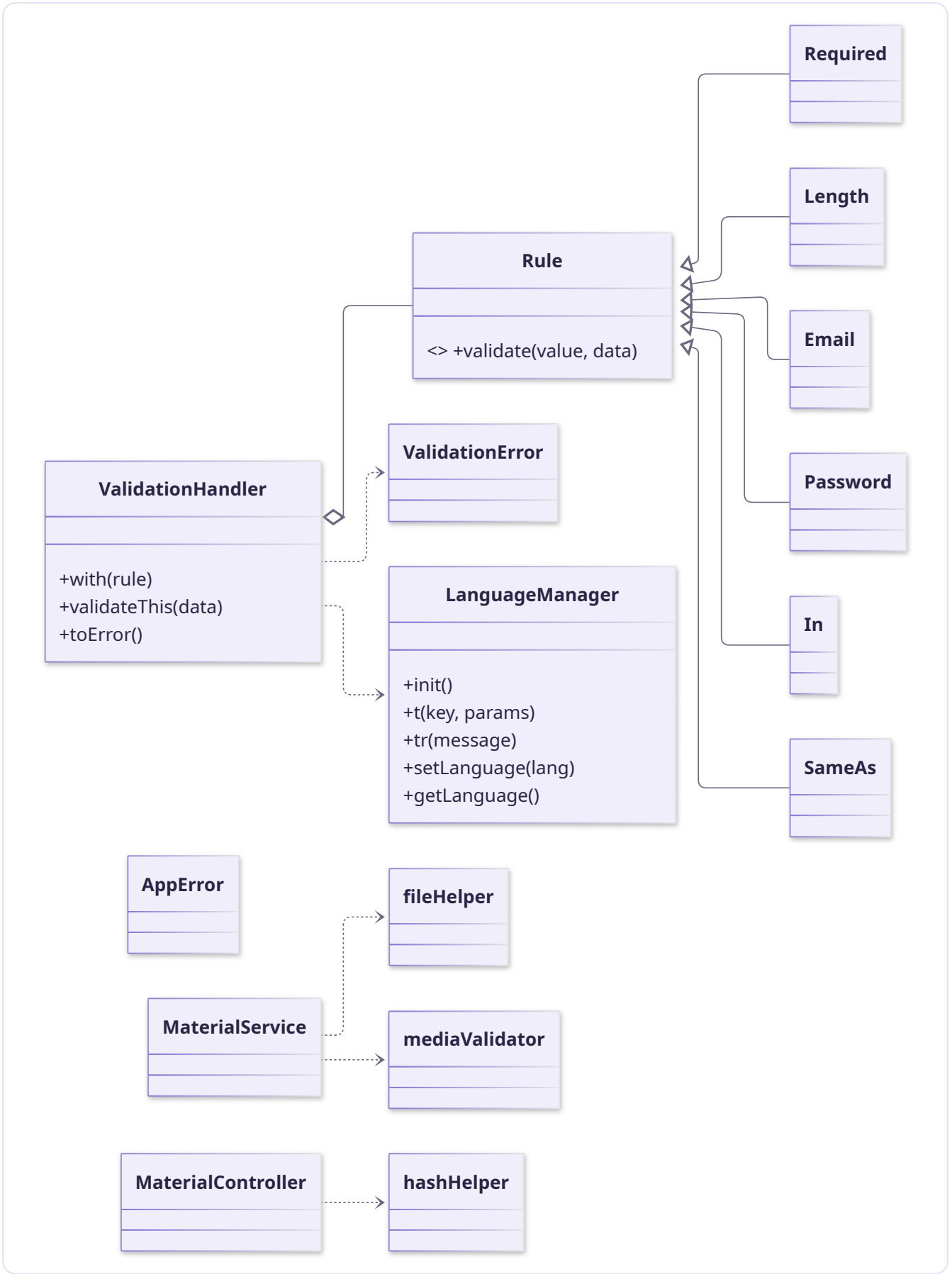
Controller factory	Key actions
`authController`	register, login, me, forgot/reset password, verify/activate, resend verification, list roles, create professor, list users, ban/unban/soft-delete.
`careerController`	list/get careers, create/update career, create/update year.
`subjectController`	list/get subjects, get subject materials, create/update subject, request/confirm/reject association.
`materialController`	global/author/mine/pending/pending-review/favorites, upload, batch, approve, reject, associate, cancel, update, delete association, delete global, favorite.
`subscriptionController`	my subscriptions, subscribe, unsubscribe, observers, ban.
`notificationController`	list inbox, mark one read, mark all read.
`auditController`	list all logs, list logs by actor.
`thematicTagController`	list, create, ban, delete.

Middleware

Component	Responsibility
`verifyToken`	Requires `Authorization: Bearer <JWT>`, verifies the signature and attaches `req.usuarioId` / `req.rol`. It does not re-query the database, so an already-issued token remains valid until expiry even if the user is later banned or soft-deleted.
`optionalAuth`	Attempts to decode a token; if absent/invalid, continues as anonymous. Used to add `esFavorito` to catalog responses.
`requireRole(...roles)`	Returns 403 unless the authenticated user's role name is in the allowed list.
`languageMiddleware`	Resolves `lang`, `x-language` or `Accept-Language` and attaches a request-local translator/language manager.
`errorMiddleware`	Converts `AppError`, validation errors and unexpected errors into localized JSON responses.
`uploadRateLimitMiddleware`	Limits upload requests per user/IP (default 50 per minute).
`rateLimitMiddleware`	Legacy duplicate upload limiter (5/minute, fixed message). It is not mounted; the server uses `uploadRateLimitMiddleware` instead.

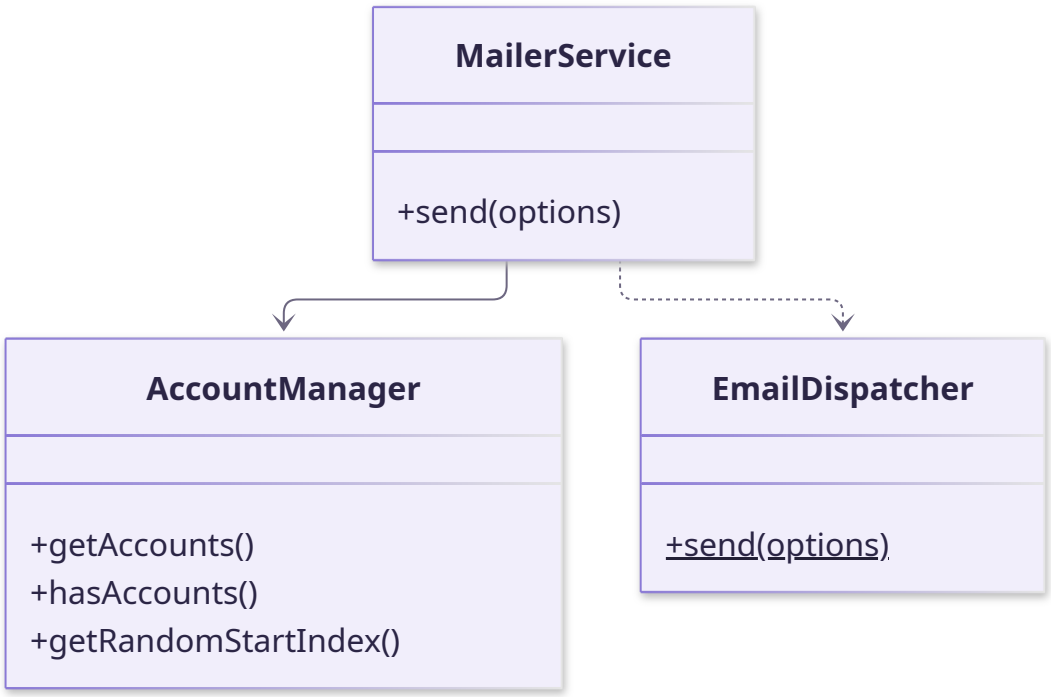
Route composition: `routes/*.js` are factories that receive the DI container, create their controller, and register endpoints. `server.js` mounts the API routers under `/api`, view routes at the root, static assets and uploads.

5.3 Validation and support classes



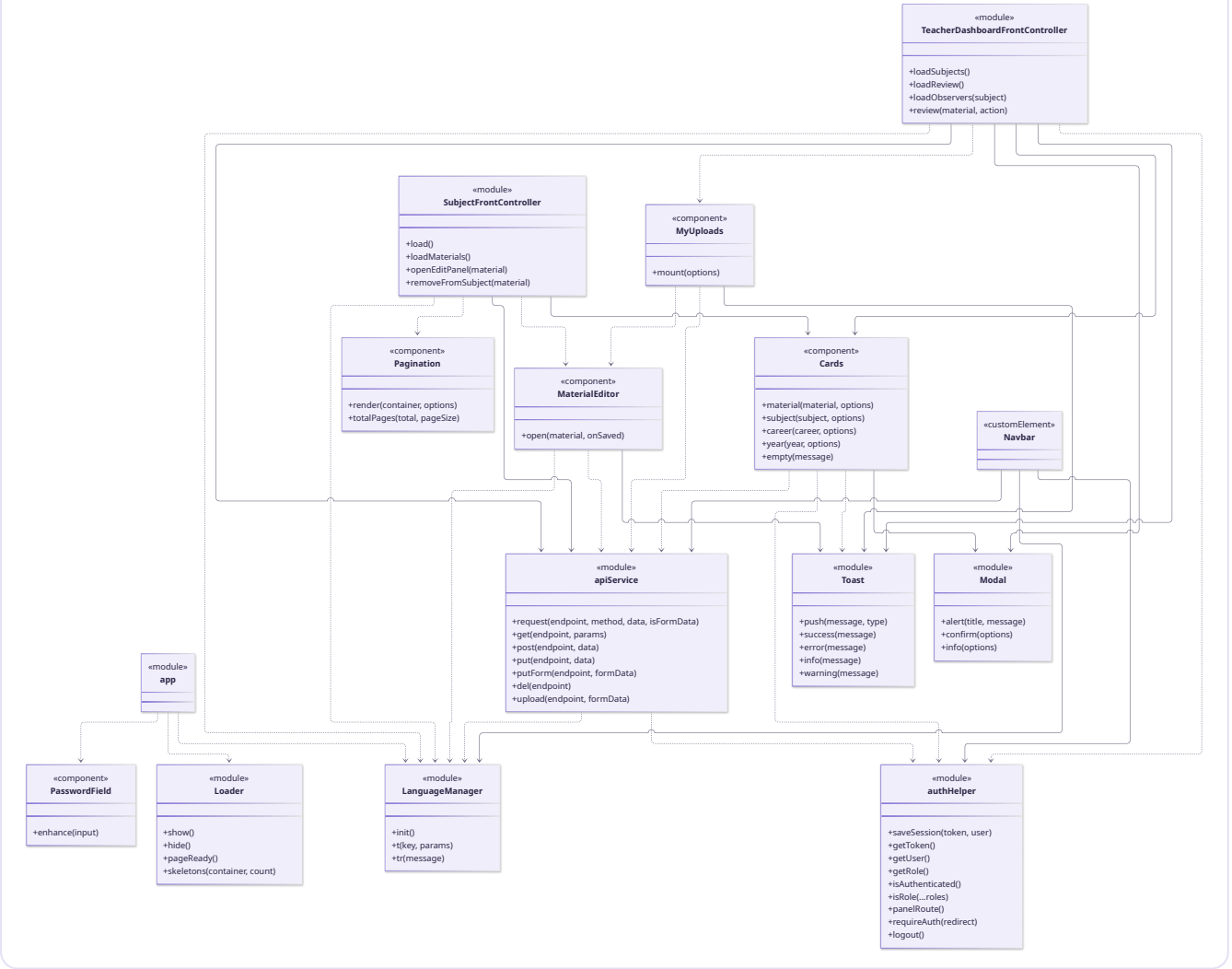
Component	Responsibility
`ValidationHandler`	Chainable validator. Each rule is applied in order; collects field errors and throws a localized `ValidationError`.
`Rule` subclasses	`Required`, `Length`, `Email`, `Password`, `In`, `SameAs`.
`ValidationError`	Carries field-level errors and an HTTP 422 status.
`LanguageManager`	Loads `es` / `en` locales, translates keys, resolves localized messages, supports parameter interpolation.
`AppError`	Application error with HTTP status and optional code/payload.
`fileHelper`	Saves/deletes files, resolves storage mode, provides URLs/paths.
`mediaValidator`	Validates file MIME/extensions and cover images.
`hashHelper`	Streams a file or string into SHA-256; used for duplicate detection.

5.4 Mail infrastructure



- `AccountManager` loads up to 10 Gmail accounts from `EMAIL_USER_n` / `EMAIL_PASS_n`.
- `EmailDispatcher` uses Nodemailer.
- `MailerService` picks a random starting account and rotates on failure.
- Auth emails are sent through `MailerService`; in non-production the API may also return a `devToken` for testing.

6. Frontend Model



Frontend module catalog

Module	File	Responsibility
`app` / `theme`	`js/app.js`, `js/theme.js`	Bootstrap locale, dispatch `bt:ready`, initialize theme, enhance password fields.
`apiService`	`js/services/apiService.js`	Fetch wrapper for `/api`; adds bearer token and `lang`, sets JSON or multipart headers, parses localized errors, logs out on protected `401` s.
`authHelper`	`js/utils/authHelper.js`	Stores token/user in local storage; role checks; dashboard redirect; `requireAuth` guard; logout.
`LanguageManager`	`js/i18n/LanguageManager.js`	Loads `es` / `en` JSON, translates keys, notifies listeners, persists language.
`authFrontController`	`js/frontControllers/authFrontController.js`	Login, register, verification notice, forgot/reset password.
`catalogFrontController`	`js/frontControllers/catalogFrontController.js`	Global catalog, filters, favorites toggle, material viewer.

Module	File	Responsibility
`subjectsFrontController`	`js/frontControllers/subjectsFrontController.js`	Subject list, career/year filters, subscription actions.
`subjectFrontController`	`js/frontControllers/subjectFrontController.js`	Subject detail, material list, upload/edit shortcuts, observer management.
`teacherDashboardFrontController`	`js/frontControllers/teacherDashboardFrontController.js`	Professor dashboard: subjects, pending review, my uploads, observer bans.
`studentDashboardFrontController`	`js/frontControllers/studentDashboardFrontController.js`	Student dashboard: subscriptions, suggestions, favorites.
`adminDashboardFrontController`	`js/frontControllers/adminDashboardFrontController.js`	Admin dashboard: users, careers, subjects, batch upload, audits, tags, associations.
`authorFrontController`	`js/frontControllers/authorFrontController.js`	Public author profile and approved materials.
`favoritesFrontController`	`js/frontControllers/favoritesFrontController.js`	Favorite list and removal.
`Cards`	`js/components/cards.js`	Material card rendering, actions, favorite button, modal viewer.
`MaterialEditor`	`js/components/materialEditor.js`	Create/edit form, file/link/cover handling, validation, tags.
`MyUploads`	`js/components/myUploads.js`	Author's upload list, status badges, edit/cancel/remove-from-subject actions.
`MaterialViewer`	`js/components/materialViewer.js`	In-app file/link preview.
`Pagination`	`js/components/pagination.js`	Client-side pagination, default page size 6.
`Navbar`	`js/components/navbar.js`	Custom element: navigation, role-aware links, language switch, notifications, logout.
`Button`, `PasswordField`, `TagInput`	`js/components/`	Reusable form/custom-element helpers.
`Modal`, `Toast`, `Loader`	`js/components/`	UI feedback and dialogs.
Frontend tests	`js/tests/`	Browser/Node test harness for API and SQL behavior.

Frontend navigation and access

- Page access is controlled client-side by `authHelper` and each page controller. There is no server-side page guard: a user can load a dashboard HTML file, but API calls still enforce roles.
- `authHelper.panelRoute()` sends `Superadmin` to `/admin`, `Profesor` to `/profesor`, and others to `/alumno`.

- `apiService` clears the session and redirects to `/login` on 401.
- The teacher dashboard filters subjects client-side with `subject.isSubjectProfessor` / `profesorIds`, matching the backend's `subject_professors`.

7. Design Patterns and Cross-Cutting Concerns

Pattern	Where	Purpose
Hexagonal architecture	Whole backend	Isolate domain/application from MariaDB and Express.
Repository	`domain/ports` + `infrastructure/repositories`	Swappable persistence.
Dependency injection	`infrastructure/container.js`	Build the object graph once; controllers/routes receive it.
Observer	`Observable`, `Subject`, `StudentObserver`	Notify subscribed students about published material.
Chain of responsibility / specification	`ValidationHandler` + rules	Compose field validations.
Factory	Controllers, routes, `buildContainer`	Create configured request handlers/routers.
Strategy-like role checks	`requireRole`, service guards	Different behavior for `Superadmin`, `Profesor`, `Alumno`.
MVC-ish	Routes/controllers → services → entities	HTTP adaptation separated from use cases.
Web Components	`Navbar`, `Button`, `PasswordField`, `Tag Input`	Reusable UI elements.
i18n observer	`LanguageManager`	Re-render/translate on language change.
Client-side pagination	`Pagination`	The API returns full result arrays; the frontend paginates.

Cross-cutting rules

Concern	Implementation
Authentication	JWT bearer token, 2-hour expiry, `verifyToken`.
Authorization	`requireRole` at route level plus ownership checks in services.
Localization	`LanguageManager` + `req.lang` /translator; API messages are translation keys resolved by `ok()` /error middleware.
Audit	`AuditLogService.record()` called by write services.
Error handling	`AppError` and `ValidationError`; localized `{ message, code, errors }`.

Concern	Implementation
File handling	`multer` → `fileHelper` / configured storage; SHA-256 duplicate detection.
Rate limiting	Upload limiter (50/minute by user/IP, overridable via env); the legacy `rateLimitMiddleware` duplicate is not mounted.
Security	bcrypt password hashes, hashed tokens, parameterized SQL via `mysql2`, safe JSON serialization.

Known implementation details / caveats

1. **Read-only controllers may bypass services.** Some listing endpoints call repositories directly to avoid duplicating filter logic. This is intentional and documented in the route reference.
2. **`ThematicTagRepositoryInterface` does not extend the base port.** Its methods throw directly; this does not affect the MariaDB implementation.
3. **`subjects.profesor_id` is denormalized.** Always treat `subject_professors` as the source of truth.
4. **Files are not automatically deleted from storage by the database.** The service calls `fileHelper`/deletes paths; a DB cascade cannot remove disk files.
5. **In-memory observer notifications are not durable.** Only rows written to `notifications` survive a request.
6. **The frontend is not a single-page application.** Each page has its own controller module and loads shared components/scripts.
7. **Client-side pagination is the current behavior.** API endpoints do not expose page/pageSize parameters except the favorites repository's internal `limit`/`offset` support, which the controller passes through without validation.
8. **Role names are exact strings.** `'Superadmin'`, `'Profesor'`, `'Alumno'`; translations only affect display, not authorization.
9. **Duplicate uploads can leave an orphan file.** A professor/student duplicate hash reuses the existing material, but the controller only cleans newly written files on error; the duplicate upload succeeds and leaves the new file on disk.